

# nginx ca reverse proxy

Teorie · o singură poartă publică în fața containerelor

Ai deja server, ai Docker, ai compose-ul cu aplicațiile tale. Ce urmează nu e o tehnologie nouă peste ele — e **răspunsul la o problemă pe care deja o ai**: în `compose-platform.yaml` publici acum **șase porturi** pe un server care stă în internet (8091 – 8095 și 3307 pe MySQL, cu `root/root`).

Lecția asta încheie cinci dintre ele și lasă una singură, cu portar în față. Nu atingi nicio linie de Java și nu reconstruiești nicio imagine.

## 1. Portul are trei părți, nu una

În notițele tale, la `creare-containarizare-docker.txt`, ai scris despre 3306: „port imagine (dockerhub) → NU IL ALEG EU”. Jumătate adevărat — și fix jumătatea cealaltă e toată lecția de azi.

```
ports:
- "127.0.0.1:8081:8081"
  A      B      C
```

| Parte | Ce e                                       | Cine o alege                                                                                   |
|-------|--------------------------------------------|------------------------------------------------------------------------------------------------|
| A     | pe ce adresă a serverului ascult           | TU — și e partea nouă de azi                                                                   |
| B     | pe ce port al serverului ascult            | TU — o faci deja: ai ales 8091, 8092, 8093...                                                  |
| C     | pe ce port ascultă programul din container | programul. La MySQL e 3306. La aplicațiile tale îl alegi tot tu, prin <code>SERVER_PORT</code> |

### CORECȚIE — la aplicațiile tale, C îl alegi tu

Regula „nu îl aleg eu” e adevărată doar pentru imagini gata făcute, ca `mysql:8.4`. La aplicațiile tale **tu îl decizi**: în `compose-platform.yaml` ai `SERVER_PORT: "8089"` și `"8093:8089"`. Ai ales și 8093, și 8089.

Contează pentru că altfel `ports:` rămâne o pereche magică de copiat, iar azi trebuie să scrii ceva ce n-ai mai scris: partea A.

### Ce face partea A

Când scrii `"8093:8089"`, Docker completează A cu valoarea implicită `0.0.0.0` — *toate* adresele mașinii, inclusiv cea publică:

```
ports: - "8093:8089"          # = 0.0.0.0:8093:8089
                                     # oricine din internet poate incerca portul
```

Când scrii explicit `127.0.0.1`, portul se leagă doar de **loopback** — interfața pe care mașina vorbește cu ea însăși:

```
ports: - "127.0.0.1:8093:8089"  # numai de pe SERVER se ajunge la el
```

De pe laptopul tău, `curl http://<IP_PUBLIC>:8093` devine `connection refused`. De pe server, `curl 127.0.0.1:8093` merge în continuare. Iar nginx rulează **pe server** — deci el ajunge, restul lumii nu.

### ASTA E TOATĂ IDEEA

Containerele nu se ascund „ștergând” `ports:` — atunci nici nginx n-ar mai ajunge la ele. Se ascund **mutându-le pe loopback**: rămân accesibile din interiorul serverului, dispar din internet.

### Excepția: MySQL

La MySQL **ștergi complet** linia `ports:`. Nu are nevoie nici măcar de loopback: singurii care vorbesc cu el sunt containerele de app, iar ele îl găsesc pe rețeaua internă Docker, după numele serviciului — exact ce faci deja cu

```
jdbc:mysql://mysql:3306/...
```

```
mysql:
  image: mysql:8.4
  # ports: - "3307:3306"    <- STEARSA. Nimic din afara compose-ului
  #                        nu are treaba cu baza de date.
```

## CE FOLOSEȘTI DEJA, FĂRĂ SĂ-I ȘTII NUMELE

Numele `mysql` din URL-ul JDBC nu e un hostname pe care l-ai configurat undeva. E **numele serviciului din compose**. Docker pornește un DNS intern în care fiecare serviciu răspunde la propriul nume, pe o rețea privată a compose-ului.

De asta baza de date merge fără niciun port publicat: aplicațiile n-au nevoie de adresa ei „din afară”, o găsesc pe dinăuntru. `ports:` nu e pentru comunicarea între containere — e doar pentru cine vine din exterior.

## „Păi Docker nu face deja asta?”

Ba da — și e util să vezi exact ce face, ca să înțelegi ce aduce nginx în plus.

```
ports: - "8093:8089"    intra pe 8093, iese pe 8089
```

Asta se numește **port forwarding**. Docker mută octeții dintr-o parte în alta fără să se uite în ei — nu știe și nu-l interesează dacă prin el trece HTTP, MySQL sau altceva. Consecința: **un port, o singură destinație**.

De asta ai acum cinci porturi publice. Port forwarding nu poate distinge două aplicații pe același port, pentru că n-a deschis niciodată plicul.

|                                                        | Port forwarding     | Reverse proxy  |
|--------------------------------------------------------|---------------------|----------------|
| Ce citește                                             | nimic — doar octeți | cererea HTTP   |
| Rutare după <code>/school</code> vs <code>/shop</code> | nu poate            | da             |
| Poate adăuga headere                                   | nu                  | da             |
| Funcționează pentru MySQL, SSH, orice                  | <b>da</b>           | nu — doar HTTP |

Ultimul rând e prețul: nginx înțelege HTTP, deci e limitat la HTTP. Port forwarding e prost, dar universal. Amândouă îți trebuie — le folosești în aceeași lecție.

## 2. Reverse proxy

Proxy înseamnă **împuțernicit** — cineva care acționează în numele altcuiva (ca la „vot prin proxy”: votează altcineva pentru tine). Singura întrebare care separă cele două feluri: **în numele cui?**

### Forward proxy — împuțernicitul clientului

Tu vrei un site. Nu ceri direct: rogi proxy-ul să ceară el, în locul tău.

```
tu —cere—> [proxy] —cere—> site.com
                        vede: proxy-ul
                        NU vede: pe tine
```

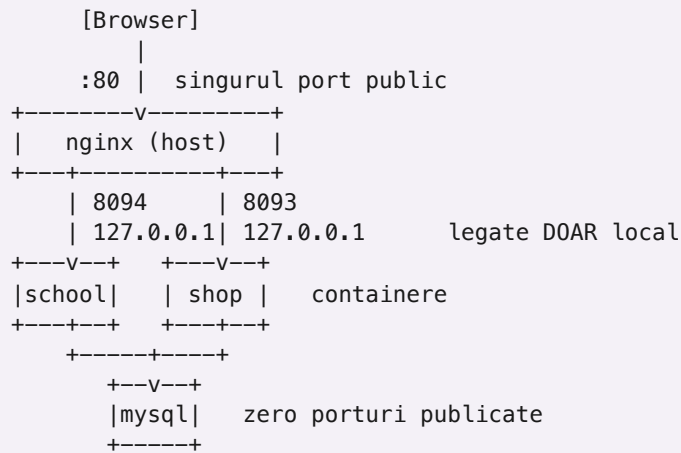
Site-ul crede că proxy-ul e cel interesat. **Tu ești ascuns**. Îl deține rețeaua ta — firma, școala, VPN-ul — și **tu ești cel configurat**: ai setat ceva în browser sau `HTTP_PROXY` în terminal. Fără acea setare, nu treci pe acolo.

### Reverse proxy — împuțernicitul serverului

Cineva vrea aplicația ta. Aplicația nu răspunde direct: nginx răspunde în numele ei.

```
browser —cere—> [nginx] —cere—> school-app
                        vede: nginx
                        NU vede: ca in spate sunt 5 aplicatii
```

Browserul crede că nginx **este** serverul. **Aplicațiile sunt ascunse** — n-au nici măcar adresă publică. Îl deții tu, operatorul, iar browserul **nu e configurat cu nimic**: tastează adresa normal și habar n-are că există un intermediar.



## Regula, în două rânduri

### CINE A PORNIT CEREREA

Vine din afară, spre serviciile mele → **reverse proxy**.

Pleacă dinăuntru, spre internet → **forward proxy**.

Ambele mută date dintr-o parte în alta. Diferă cine a pornit-o.

Când direcția e greu de spus, mai există un test care nu dă greș: **a trebuit configurat clientul ca să treacă pe acolo?** Da → forward. Nu → reverse. Forward proxy funcționează doar pentru cine l-a setat explicit; reverse proxy funcționează pentru toată lumea, fără ca cineva să știe de el.

### DE UNDE VINE CONFUZIA DIN NUME

„Forward proxy” a apărut primul, istoric — proxy-ul de firmă. Când aceeași unealtă a fost pusă de **cealaltă parte** a conexiunii, i s-a spus „reverse” = *invers față de cel clasic*.

Numele descrie o comparație istorică, **nu direcția traficului**. „Reverse” nu înseamnă „trimite înapoi” sau „iese în afară” — traficul merge înainte în ambele cazuri. Și de asta „proxy” spus simplu, fără adjectiv, înseamnă aproape întotdeauna *forward proxy*.

## Ce câștigi

- **Un singur port public.** Restul serviciilor nu mai există pentru internet.
- **Un singur loc pentru HTTPS** mai târziu — nu în fiecare aplicație.
- **Adrese stabile.** Muți o aplicație pe alt port intern, adresa publică rămâne aceeași.
- **Un singur log** cu tot traficul care intră.

## 3. De ce nginx pe server, nu în container

Se poate și în container. Pe serverul tău, acum, nu merită:

- **Se instalează cu apt** și devine un serviciu obișnuit — `systemctl status/reload nginx`, la fel cum ai făcut cu Docker.
- **Log-urile stau la locul lor**, în `/var/log/nginx/`, nu în spatele lui `docker logs`.
- **Un intermediar mai puțin de debugat.** Când o cerere nu ajunge, ai două verigi de verificat (nginx → container), nu trei.
- **Ajunge la containere prin loopback**, exact adresa pe care le-ai legat la punctul 1.

Regula de reținut: nginx în container are sens când *totul* e în containere și nu mai administrezi serverul de mână. Ajungem și acolo — dar are capcane proprii, și le merită o lecție separată.

## 4. sites-available, sites-enabled și „nginx -t”

Pe Ubuntu, configurația nginx e împărțită în două foldere:

| Folder                      | Ce ține                                                                                           |
|-----------------------------|---------------------------------------------------------------------------------------------------|
| /etc/nginx/sites-available/ | toate configurațiile scrise, active sau nu                                                        |
| /etc/nginx/sites-enabled/   | doar cele active — și nu copii, ci <i>symlink-uri</i> către cele din <code>sites-available</code> |

Un site se activează făcând legătura și se dezactivează ștergând-o — fișierul rămâne scris. Nu-l rescrii de la zero când vrei să-l repui.

```
sudo ln -sf /etc/nginx/sites-available/platform \
    /etc/nginx/sites-enabled/platform # activeaza
sudo rm /etc/nginx/sites-enabled/default # dezactiveaza default-ul
```

Fără ultima linie, pagina „Welcome to nginx!” continuă să răspundă pe `/`, pentru că e și ea un site activ.

#### REGULA — validezi ÎNAINTE de reload

```
sudo nginx -t && sudo systemctl reload nginx
```

`nginx -t` citește configurația și spune dacă e validă, **fără** s-o aplice. Cu `&&`, reload-ul pornește doar dacă testul a trecut.

E același reflex ca `docker compose config` înainte de `up` — cu diferența că aici, dacă greșești, cade site-ul de pe un server public.

#### reload vs restart

|                   | reload                               | restart                                              |
|-------------------|--------------------------------------|------------------------------------------------------|
| Conexiuni în curs | rămân servite până se termină        | se taie                                              |
| Config invalid    | refuză și <b>păstrează ce mergea</b> | oprește, apoi nu mai pornește — site căzut           |
| Când îl folosești | de fiecare dată                      | doar dacă ai schimbat ceva la serviciu, nu la config |

Reflexul corect e `reload`. `restart` pe o configurație greșită e felul clasic în care cade un server la o modificare de doi lei.

### 5. Context path — de ce Swagger se rupe fără el

nginx trimite cererea pentru `/school/api/courses` la containerul `school`. Dar aplicația ta, care nu știe nimic despre nginx, își caută rutele la `/api/courses` — fără prefix. Rezultatul: **404**.

Și mai vizibil la Swagger: pagina se încarcă, dar butoanele „Try it out” trag spre `/api/...` în loc de `/school/api/...`, pentru că Spring generează link-urile după ce crede *el* despre adresa lui.

Se rezolvă spunându-i lui Spring pe ce prefix trăiește:

```
environment:
  SERVER_SERVLET_CONTEXT_PATH: /school
```

#### DE CE MERGE FĂRĂ SĂ RECONSTRUIEȘTI IMAGINEA

Spring citește orice proprietate și din variabilele de mediu, transformând `SERVER_SERVLET_CONTEXT_PATH` în `server.servlet.context-path` — majuscule și liniuțe de subliniere în loc de puncte. Se numește *relaxed binding*.

**Îl folosești deja:** `SERVER_PORT: "8089"` din `compose-ul` tău merge pe exact același mecanism. De asta lecția asta nu cere niciun `mvn package` și niciun `docker push` — se schimbă doar stratul de ops din jurul aplicației.

Cu context path setat, adresa e coerentă cap-coadă: browserul cere `/school/...`, nginx trimite `/school/...`, iar Spring știe că prefixul lui e `/school` și potrivește restul.

## Headerele de proxy

```
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

Consecința directă a faptului că nginx răspunde în numele aplicației: aplicația nu mai primește cererea de la utilizator, ci **de la nginx**. Fără liniile astea, Spring vede toate cererile venind de la `127.0.0.1` — în log-uri, o mie de utilizatori diferiți arată ca unul singur, de pe aceeași adresă.

`X-Forwarded-For` e nginx spunându-i aplicației: „*eu ți-o transmit, dar a cerut-o cineva de la adresa X.*” La un forward proxy linia n-ar avea sens — acolo scopul e chiar ca serverul să **nu** afle cine a cerut.

## 6. Firewall-ul AWS (Security Group)

Security Group e un firewall **în fața** mașinii: ce nu e permis acolo nu ajunge niciodată la ea. După lecția asta îți trebuie exact două reguli:

| Port      | Sursă    | De ce                                 |
|-----------|----------|---------------------------------------|
| 22 (SSH)  | My IP    | ca să te conectezi tu — nu „Anywhere” |
| 80 (HTTP) | Anywhere | poarta publică, nginx                 |

Porturile `8091` – `8095` și `3307` se scot. Observă ordinea: le-ai mutat deja pe loopback la punctul 1, deci **chiar dacă cineva le-ar redeschide în firewall, tot n-ar răspunde**.

### DOUĂ STRATURI, NU UNUL

Security Group oprește traficul *înainte* de mașină. Legarea pe `127.0.0.1` oprește traficul *în* mașină. Sunt independente — și de aia se pun amândouă: o greșeală într-unul nu deschide singură ușa.

## Cum verifici, la final

Trei comenzi. De pe **laptop**, nu de pe server:

```
curl http://<IP_PUBLIC>/school/swagger-ui/index.html # HTML
curl http://<IP_PUBLIC>:8093                          # refused
curl http://<IP_PUBLIC>:3307                          # refused
```

Prima trebuie să meargă, ultimele două trebuie să **nu** meargă. Dacă toate trei răspund, lecția nu e făcută — e doar instalată.